

I left a MySQL server open to the internet.

Two threat actors found it in **4 hours**, wiped every database, and left a ransom note demanding **0.0135 BTC (~\$11K CAD)**. They didn't break in with fancy exploits, they simply logged in.

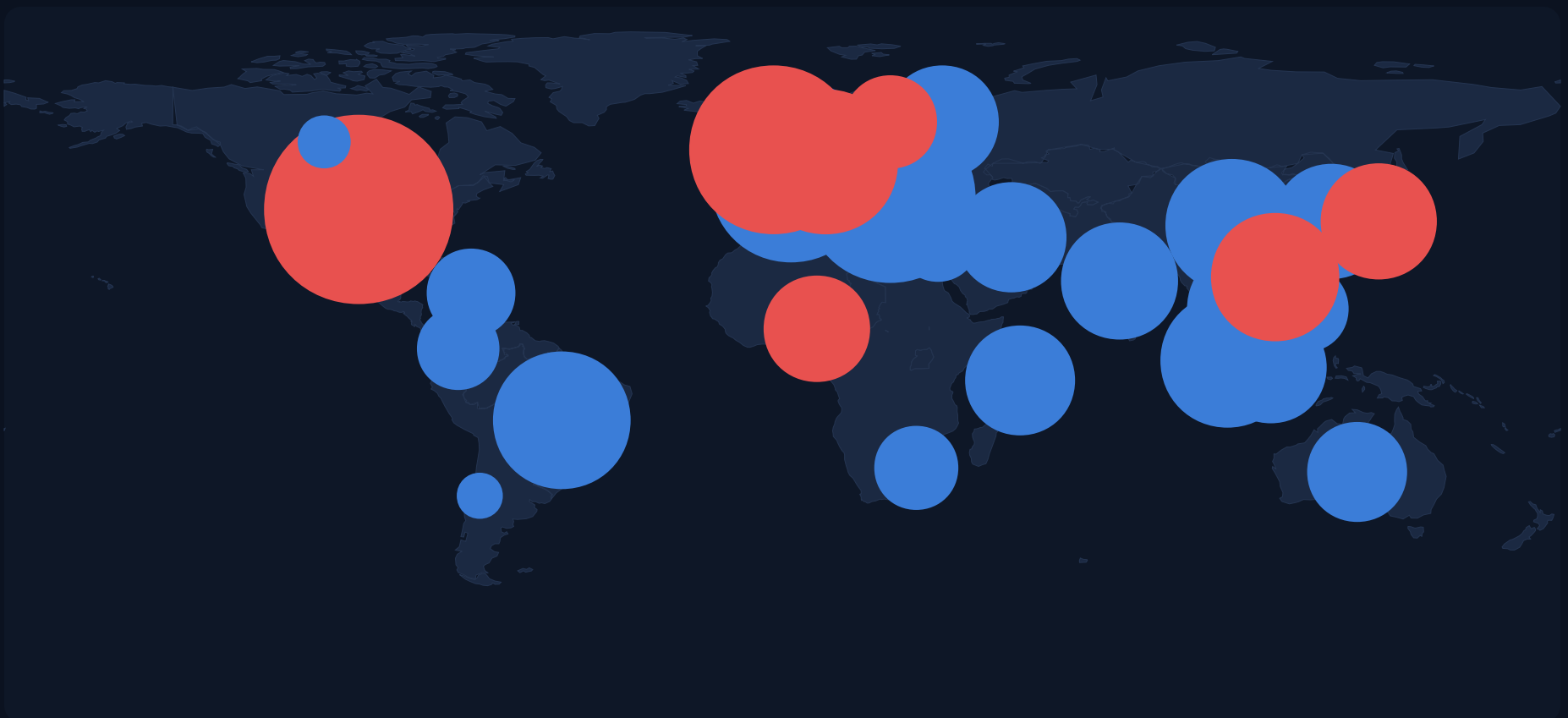
- Ransom Note from 'RECOVER_YOUR_DATA' table in MySQL Database

All your data was backed up by us. You must pay **0.0135 bitcoin** to `bc1qa83x6l2dlgkx7cc9rmrymscp5sa3ljepu42w2r` or in 48 hours, your data will be publicly disclosed and deleted. After payment send mail to `ak+218oj@onionmail.org`.
Your **DATAID** is: 2180J

THE EXPOSURE

7,435 IPs.
97 countries.
4 days.

I deliberately added an NSG rule named `danger-allow-all-inbound` to attract real adversary traffic. **34 alerts** fired from my custom Microsoft Sentinel and Defender XDR rules.



■ Confirmed attacker infrastructure ■ Scanning / probe traffic

1,829

inbound hits on RDP (3389)

181

inbound hits on MySQL (3306)

The attackers' dwell time in my environment was 4 days – by choice. I intentionally let the adversary perform harmful actions on objective to observe their behavior using Defender XDR. In a production SOC, this incident would be contained within minutes.

Two attackers. Mapped to MITRE ATT&CK.

Two independent adversaries broke in through the same exposed host VM.

Reconnaissance	T1595 Active Scanning	7,435 source IPs from 97 countries scanned the exposed ports.
Initial Access	T1190 Public-Facing App T1078 Valid Accounts	The 'danger-allow-all-inbound' rule exposed MySQL (3306) and RDP (3389). Successful authentication as 'root' and 'administrator' with legitimate login credentials from 11 external IPs.
Credential Access	T1110 Brute Force	77 failed MySQL and 90 failed RDP attempts before the successful logons.
Collection	T1005 Data from Local Sys	'SELECT' queries used to read PII from the MySQL database (SSN, DOB, credit card).
Exfiltration	T1041 Exfil over C2	505 KB of data exfiltrated to external IP '64.89.163.179'.
Impact	T1485 · T1531 T1657	Dropped 3 databases, locked out the administrator via REVOKE, and planted 4 ransom notes demanding payment in bitcoin.

WHAT THE ATTACKER DID

96 seconds from login to total loss

MySQL root - no exploit, no malware, no privilege escalation.

- | | |
|-----------|--|
| 06:30:07 | Logs in as root from a US IP. First try. |
| 06:31:37 | Reads the sensitive columns: SSN, DOB, card data, credentials |
| ~06:43 | Exfiltrates 505 KB - ~8,600× the median flow |
| THE PROOF | |
| 06:32:08 | DROP DATABASE ×3 - all data gone |
| 06:32:10 | Creates RECOVER_YOUR_DATA - demands 0.0135 BTC |
| 06:32:12 | REVOKE on root - blocks my own recovery |

The attackers *REALLY* exfiltrated my dummy data.

Flow data (NTANetAnalytics) proved the ransom note was not a bluff. This is important to consider for PROD environments with real data on the line (not dummy data).

The trap, the tripwire, and the cleanup

Every misconfiguration below was deliberate.

1 Weaken the database (place the bait)

Exposed ports 3306 (MySQL) and 3389 (RDP) to the public internet.

Weak password on the MySQL network root (not local root) account - 'root'@'%'.

2 Wrote 2 Sentinel detection rules

For Successful MySQL logon (parsed from raw audit data, with failed-connection correlation to kill false positives), and for successful VM logon on admin / guest users.

3 Both alerts fired in Defender XDR during the live intrusion

34 alerts across 2 incidents. Alert suppression settings reduced noise, and the detections fired while the attack was taking place.

4 Contained the incident (remediation)

Isolated the VM in Defender XDR the moment I was ready to close the exercise.

5 Eradicated & recovered

Killed the NSG rule → rotated the root credential → deleted `administrator`, disabled guest → restored the databases from backup.

Successful SQL Server Logon

Parses raw MySQL audit data and correlates against failed connections to suppress false positives. **32 alerts → 1 incident.**

```
// Ash - Successful SQL Server Logon - corp-win-na01shy [Medium]
let MyDevice = "corp-win-na01shy";
let FailedConnections =
    MySQLAudit_CL
    | extend RawData = replace_string(RawData, "\t", " ")
    | extend DeviceName = tostring(split(_ResourceId, "/")[-1])
    | where DeviceName == MyDevice
    | where RawData has_any ("Access denied", "Lost connection",
        "does not support authentication protocol", "Aborted connection")
    | extend ConnectionId = extract(@"^S\s+(\d+)\s+Connect", 1, RawData)
    | distinct ConnectionId;
MySQLAudit_CL
| extend RawData = replace_string(RawData, "\t", " ")
| extend DeviceName = tostring(split(_ResourceId, "/")[-1])
| where DeviceName == MyDevice and RawData has "Connect"
| extend ConnectionId = extract(@"^S\s+(\d+)\s+Connect", 1, RawData)
| extend ActionType = case(
    RawData has_any ("Access denied", "Lost connection", ...), "LogonFailure",
    ConnectionId in (FailedConnections), "Ignore",
    RawData !has "@", "Ignore", "LogonSuccess")
| where ActionType == "LogonSuccess"
| project TimeGenerated, DeviceName, Username, IPAddress, ActionType
| order by TimeGenerated desc
```

Successful VM logon

Flags interactive/network logons to the host on the administrator or guest accounts. **2 alerts → 1 incident.**

```
// Ash - Successful VM Logon - corp-win-na01shy [Medium]
DeviceLogonEvents
| where DeviceName has "corp-win-na01sh"
| where AccountName in~ ("administrator", "guest")
| where ActionType == "LogonSuccess"
| project TimeGenerated, RemoteIP, AccountName,
           DeviceName, ActionType, LogonType
| order by TimeGenerated desc
```

Full incident report available on GitHub ↓

<https://github.com/ashiqs1/azure-mysql-honeypot>